



Intro to ITECH

By the end of Week 1...

By the end of Week 1, you should be able to:

1. Understand the **types of web applications**
2. Understand **tiered architectures** and their benefits
3. Interpret and create **high-level architecture diagrams**
4. Understand **data modeling** and create **ER diagrams**
5. Apply Information Architecture principles to develop **user requirements, sitemaps, user flows, and wireframes**

This lecture covers most of what you need for the first part of your group coursework!

**WHAT IS A WEB
APPLICATION?**

Web Applications Examples



your first stop for train tickets

What is a Web Application?



Software that you use on your **browser**



Has two parts: the page you see (**frontend**) + code on a server (**backend**)



The frontend and backend **communicate over the web** (requests and responses)



Stores and loads data (often from a database)



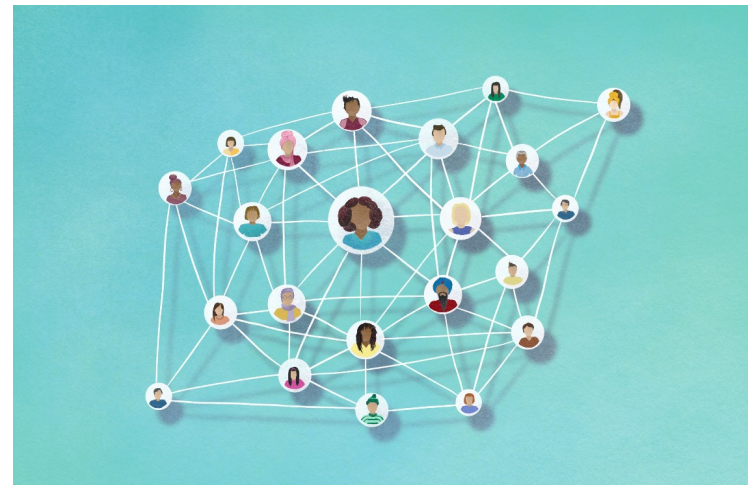
Many users can use it at the same time, who need to access timely and useful information

Internet vs Web



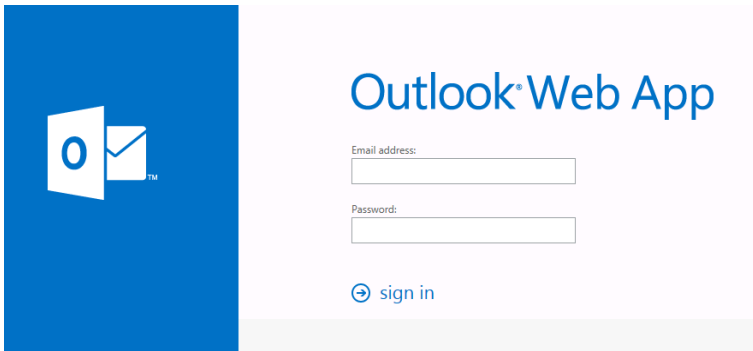
Internet: the network connecting computers

Web: what browsers use on that network – links, pages, web applications



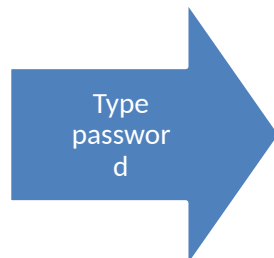
Frontend vs Backend

**Frontend: what you see
(text, images, buttons,
forms)**



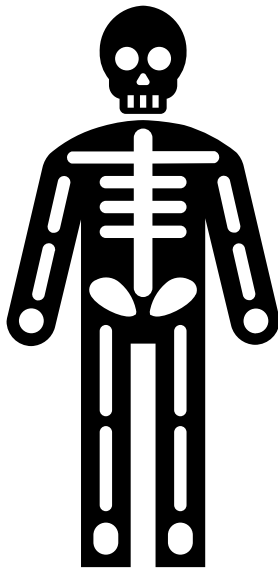
ThePhoto by PhotoAuthor is licensed under CCYSA.

**Backend: what happens
(rules, processing,
security, data)**

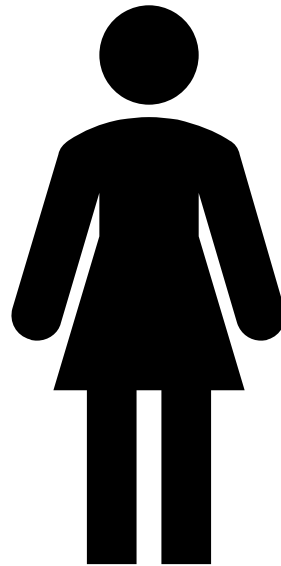


Frontend Building Blocks

**HTML =
content**



**CSS =
appearance**



**JavaScript =
behavior**



What is a Web Application? (some jargon)



It is a distributed information management system (DIM)



Enables management, sharing, finding and presentation of information over the web



Does so over a network, in a distributed fashion



Typically has many users, often geographically separated



Ideally DIMs enable users to access timely, relevant and useful information in a seamless manner

Types of Web Apps (1)

- **Static**

- Originally web applications were just collections of hand-built HTML pages
- These required manual update and introduced the possibility of inconsistency

- **Interactive**

- Forms, selection menus and radio buttons on web pages offer the possibility of selectively chosen pages generated by server-side programs
- CGI was the first technology but has been superseded by others (ASP, JSP, PHP, Cold Fusion, etc.)
 - e.g. an online survey site

- **Transactional**

- Forms also offer the capture of data and database storage at the server
- Although the data management may be separated from the internet server
 - e.g. e-commerce checkout/payment process

- **Workflow-based**

- Support for functionality express as a sequence of pages reflecting a business process
 - e.g. hotel booking, flight booking

Types of Web Apps (2)

- **Portal Oriented**

- One point of access is given to multiple web sites
 - e.g. Virtual Shopping malls

- **Collaborative**

- Web sites which permit multiple users to share information management
 - e.g. Wikis such as Wikipedia

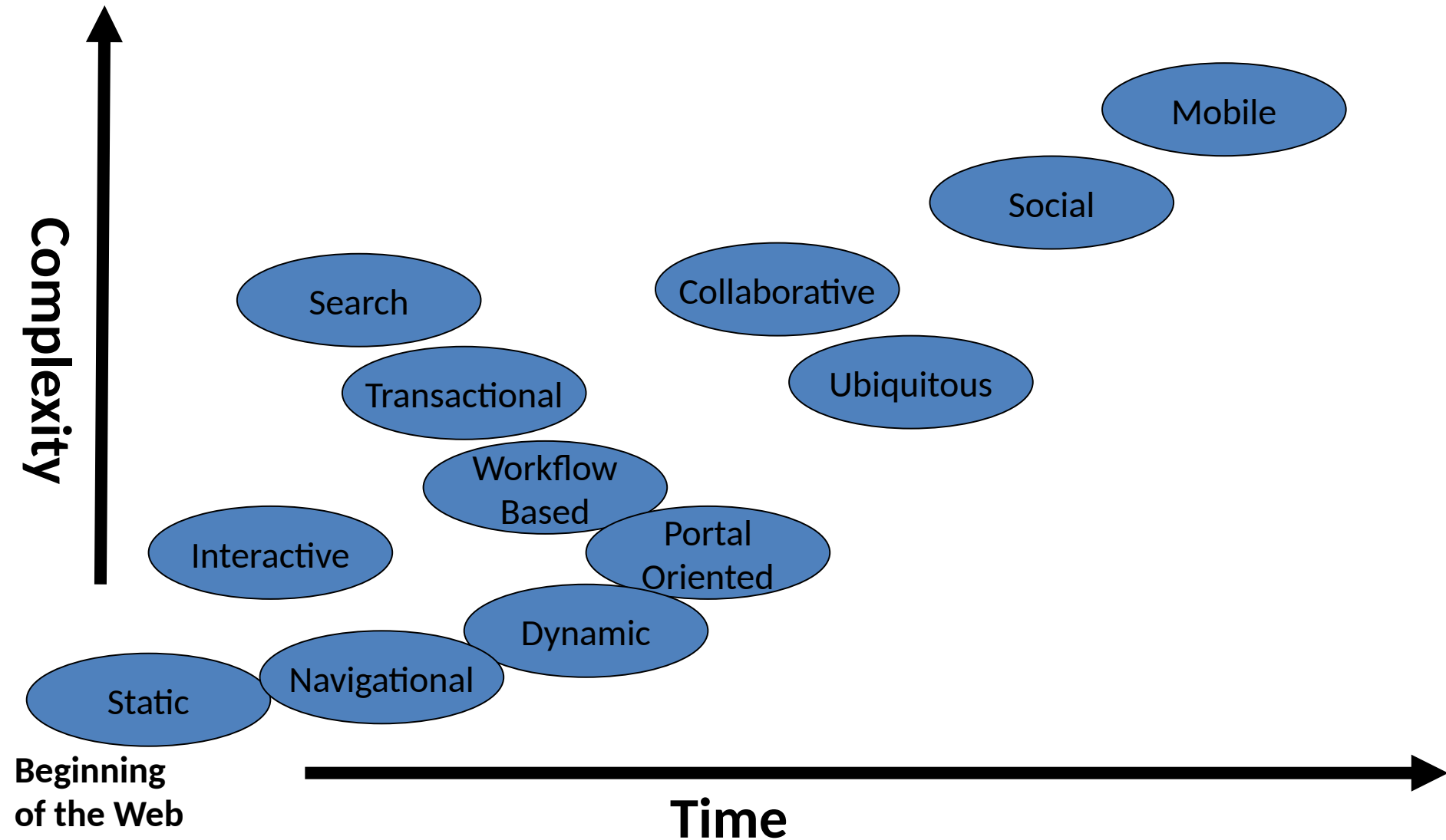
- **Social Web**

- Many sites provide a focal point for communities
 - e.g. Photo sharing sites, Facebook, etc.

- **Mobile and Ubiquitous**

- “Web” applications increasingly provide access by small, mobile and non-visual devices (i.e. phones) as well as data capture by sensors
 - e.g. Maps applications

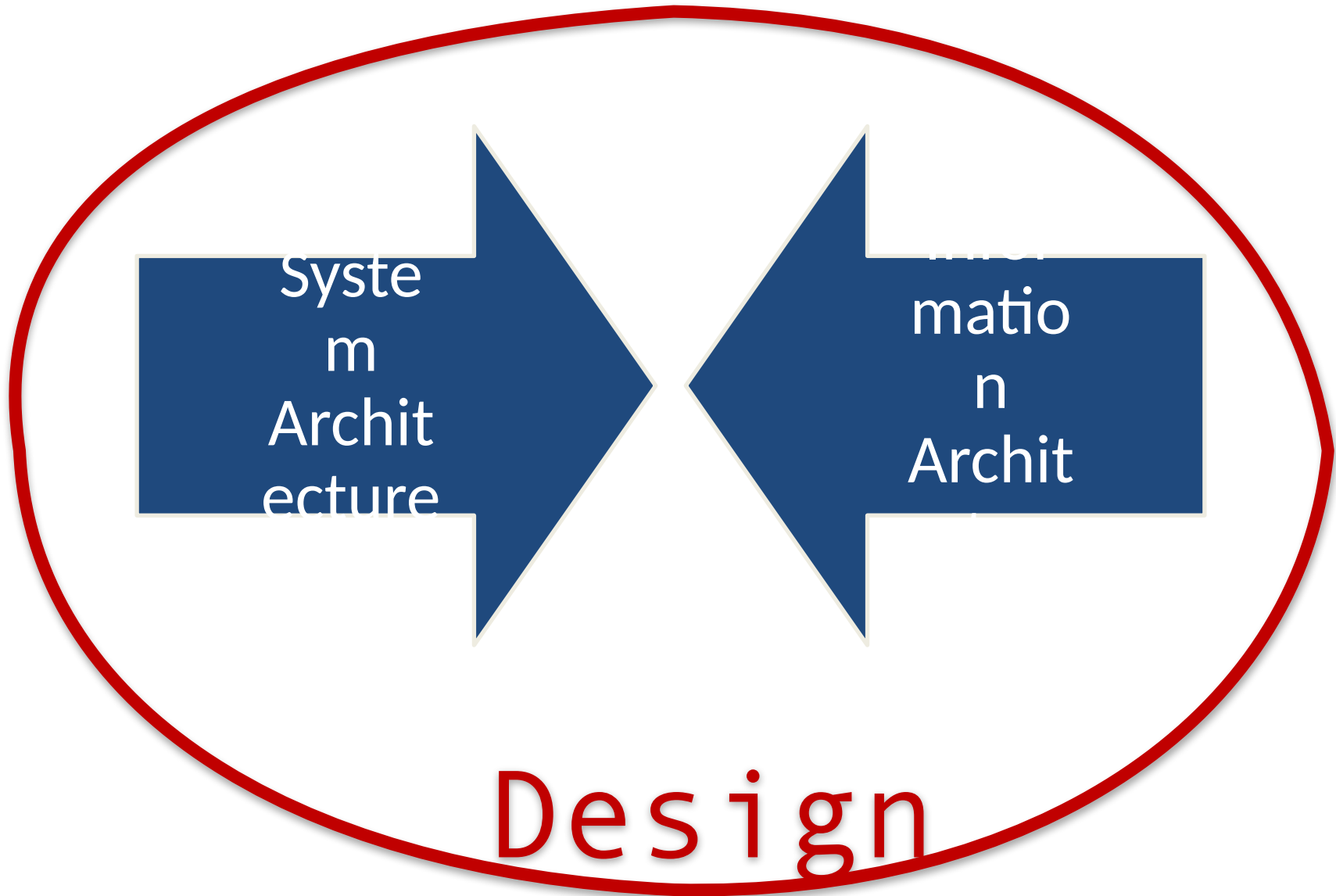
Web App Evolution



Web Complexities

- **The nature and structure of the application**
 - The hypertext structure
 - The presentation and layout
 - Content usage and management
 - Service usage
- **The users and usage of the application**
 - Instant online access, with permanent availability
 - A wide variety of usually anonymous users
 - On a variety of devices
- **The development process behind the application**
 - by a multidisciplinary team
 - In a state of continuous development
 - A mixture of legacy technologies and immature technologies
- **Understanding Website Complexity?**
 - <https://web.eecs.umich.edu/~harshavm/papers/imc11.pdf>

Need for principled and disciplined approach...



Architecture Design

- System Architecture (system focused)
 - Specifications and Technical Requirements
 - High Level System Architecture
 - Sequence Diagrams
 - Entity Relationship Diagrams
- Information Architecture (user focused)
 - User Requirements
 - Site maps
 - User Flows / Journeys
 - Wireframes

We will learn how to produce a number of these design elements during the course. They are also a significant component of your group project.

Design and Diagrams

- Architects need to communicate to developers how the application and its components fit together and who is responsible for what
- The designs also serve as a communication tool with the client
- It is important to be able to draw and read such diagrams especially when projects become large and complex

SYSTEM ARCHITECTURE

System Architecture

- Every **component** of a system must be **designed**
- The **architecture** of the **system** shows the **blueprint** or plans of **how each component fits together**.
- Architectural diagrams can be at various levels
 - Data structure / algorithm / object level
 - Component / library level
 - Application level
- Various diagrams are needed

We focus on the application level or the high-level system architecture

Tasks of an application

- The main tasks that any application must support:
 - **user interface** management
 - the implementation of **algorithms** – the business logic
 - **information manipulation**
 - **data storage**

Monolithic Programs

- Example: A monolithic program in JAVA performs all the tasks
 - user interface with SWING
 - algorithms in methods
 - information manipulation in methods, e.g. sort methods
 - data storage using File I/O
- This reflects a 1-tier architecture – not suitable for web applications!

Tiered Architectures

- The **structure of applications** have changed from:
 - **Monolithic** programs in which every aspect of the application is coded
 - To **tiered** structures in which different aspects are separated into different levels
- The **advantage** of the **tiered architectures** are
 - Each tier can be coded **separately** without the programmer having to deal with everything
 - The different tiers can be **distributed over the network** leading to increased efficiency
- The **tiers** must **interact effectively!**
 - There must be well defined interfaces between adjacent tiers
 - Internet protocols and database connection software do this well

TWO-TIER ARCHITECTURES

Data Management (1)

- Data and Information Management is common to many applications
 - Handling and dealing with large amounts of data is required by many applications
 - Access to a shared repository of data is highly beneficial to facilitate information flows between actors
 - Storing, retrieving, modifying and securing are common to many application
 - Databases Systems and Information Retrieval Systems provide ways to manage this data and information (efficiently)

Data Management (2)

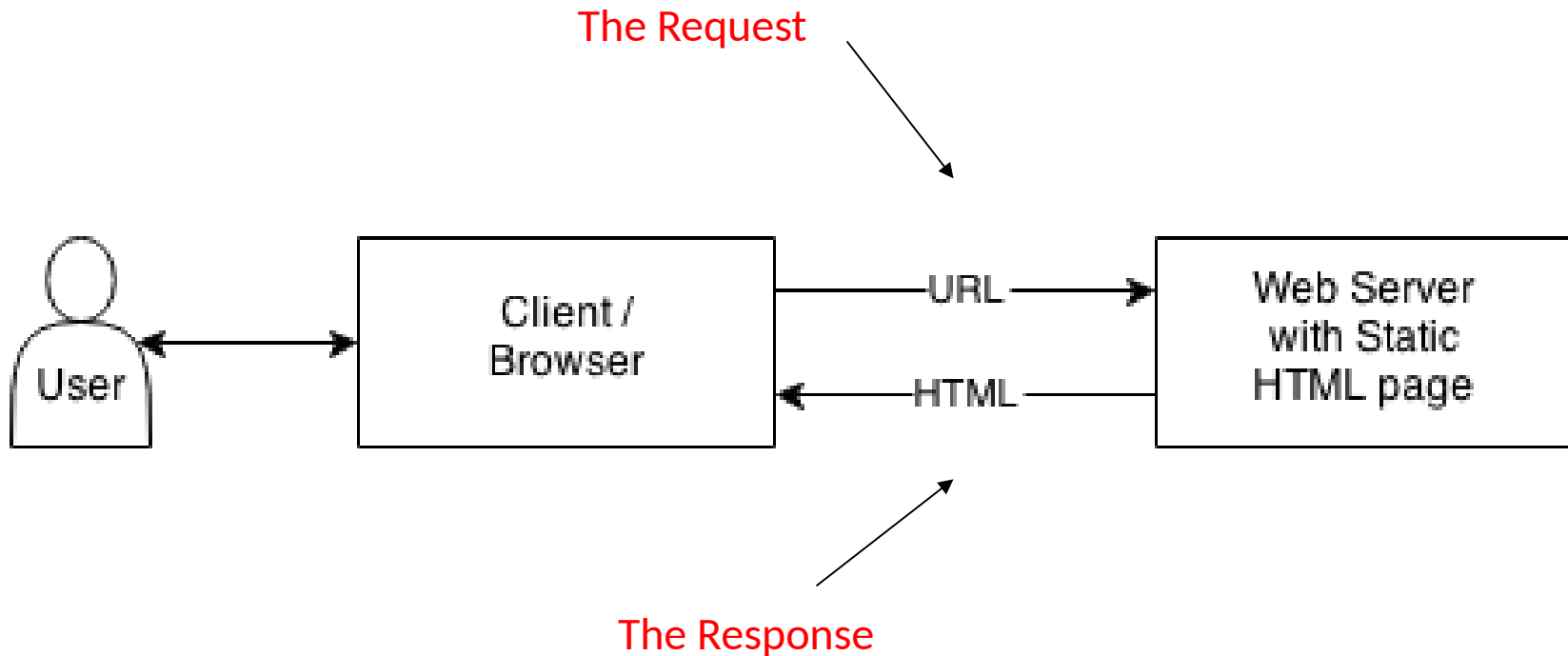
- It is **hard/difficult/time-consuming** to write the code which accesses large amounts of data efficiently
- **Solution: Separate the concerns**
 - Let **Database Systems** to handle the data management
 - And then use a **Client application** to interact with the database:
 - The client acts like a database user sending in queries and updates and making use of the result
 - The standard mechanism is to code **SQL statements** as strings inside the program

Static Web Application

- Static web sites consist of a set of hyper-linked HTML files
- Each HTML file describes one page of the web site
- Each page includes one or more links to other pages
- If a user clicks a links or enters a URL then a request is sent to the web server identifying the next page to load

Two Tier Architecture

Thin Client: This is the standard architecture for serving up static content on the web.

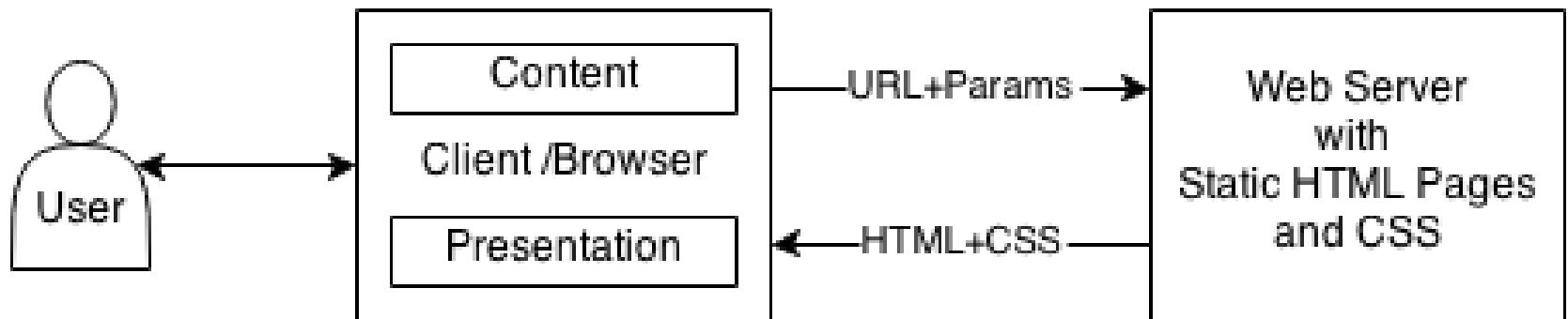


Style and Substance

- Initially, the HTML files contained all the information on how to render the page
 - The text/data (html) and the style (css),
- With lots of pages there is lots of overhead in creating and maintaining the site.
- **Solution: Separate the concerns**
 - Use external Cascading Style Sheets (CSS) which act upon the different elements in the html.
 - The web server/application returns a html file and css file.
 - i.e. one to hold the Content and one to represent the Style

Two Tier with Layers

Extended Thin Client: Layers within the client tier handles the content and look/feel.



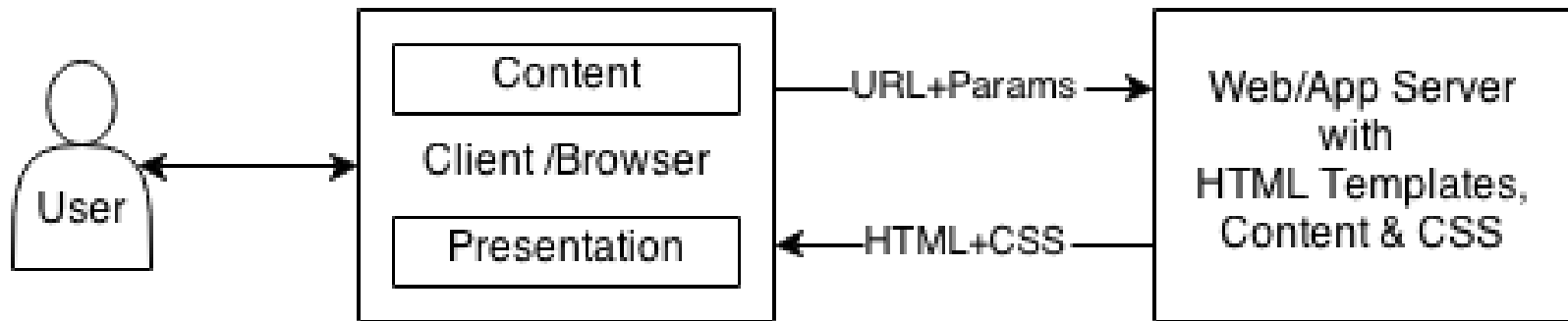
The **content layer** represents/stores the data

The **presentation layer** renders the data

Two Tier with Layers & Servers

Extended Thin Client and Extended Server:

Layers within the client tier handles the content and look/feel. The server now houses a web server and an application server.



Web Server handles incoming requests and send outgoing responses, routing them to/from the correct application server

Application Server is typically a script that dynamically generates a response given a request

Two Tier Issues

- Distributed applications with a **two-tiered architecture** distinguish only:
 - **client** processes – control the application logic and the user interface
 - **server** processes – supply resources, such as data, to the clients
 - however this:
 - puts a lot of the load on the client
 - ties the client software to the database software so changes in each affect the other

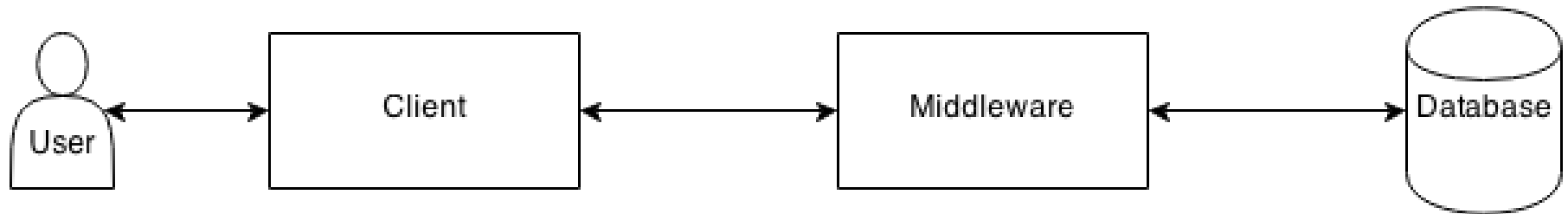
Further Tiered Architectures

- An improvement is to separate the application into **three (or N-) tiers**:
 - **presentation** processes deal exclusively with the user interface – this might, for instance, be through the use of a browser or other user agent
 - **application** processes deal with the logic of the application, queries, calculations, etc. – there is either one (3-tier) or more (N-tier) of such processes
 - **data source** processes supply the data from a database (binary) or a file (for instance, XML)

3- TO N-TIER ARCHITECTURES

Three Tier Architecture

Basic Three Tier Architecture: where the client handle the interaction with the user, the middleware handles the application logic, and the database stores the data.



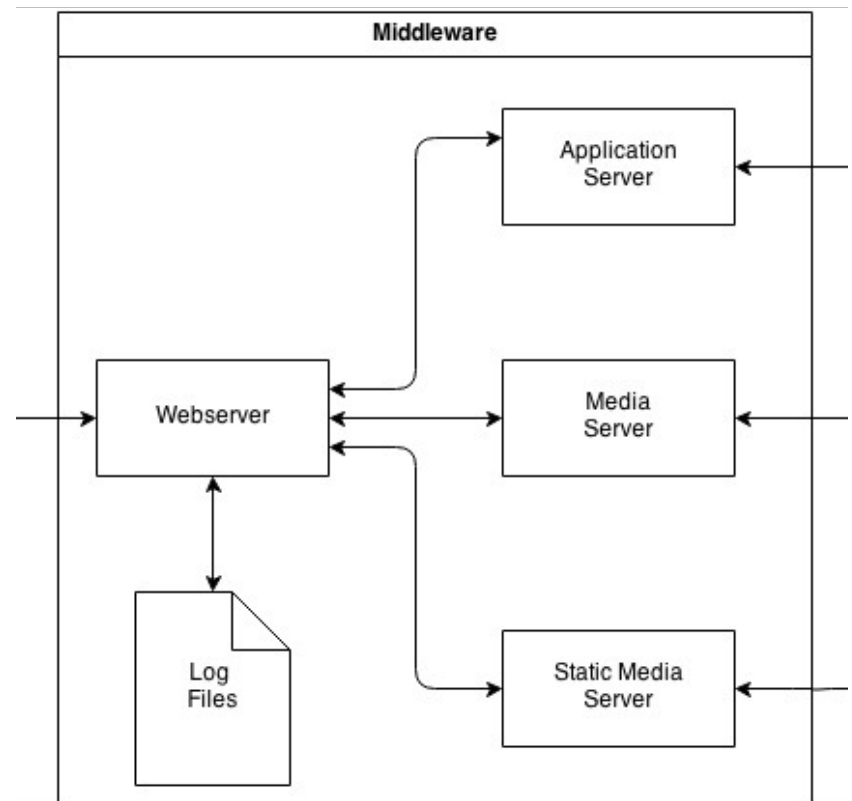
Database: persists and manages the data associated with the application.

Middleware (Zoomed in)

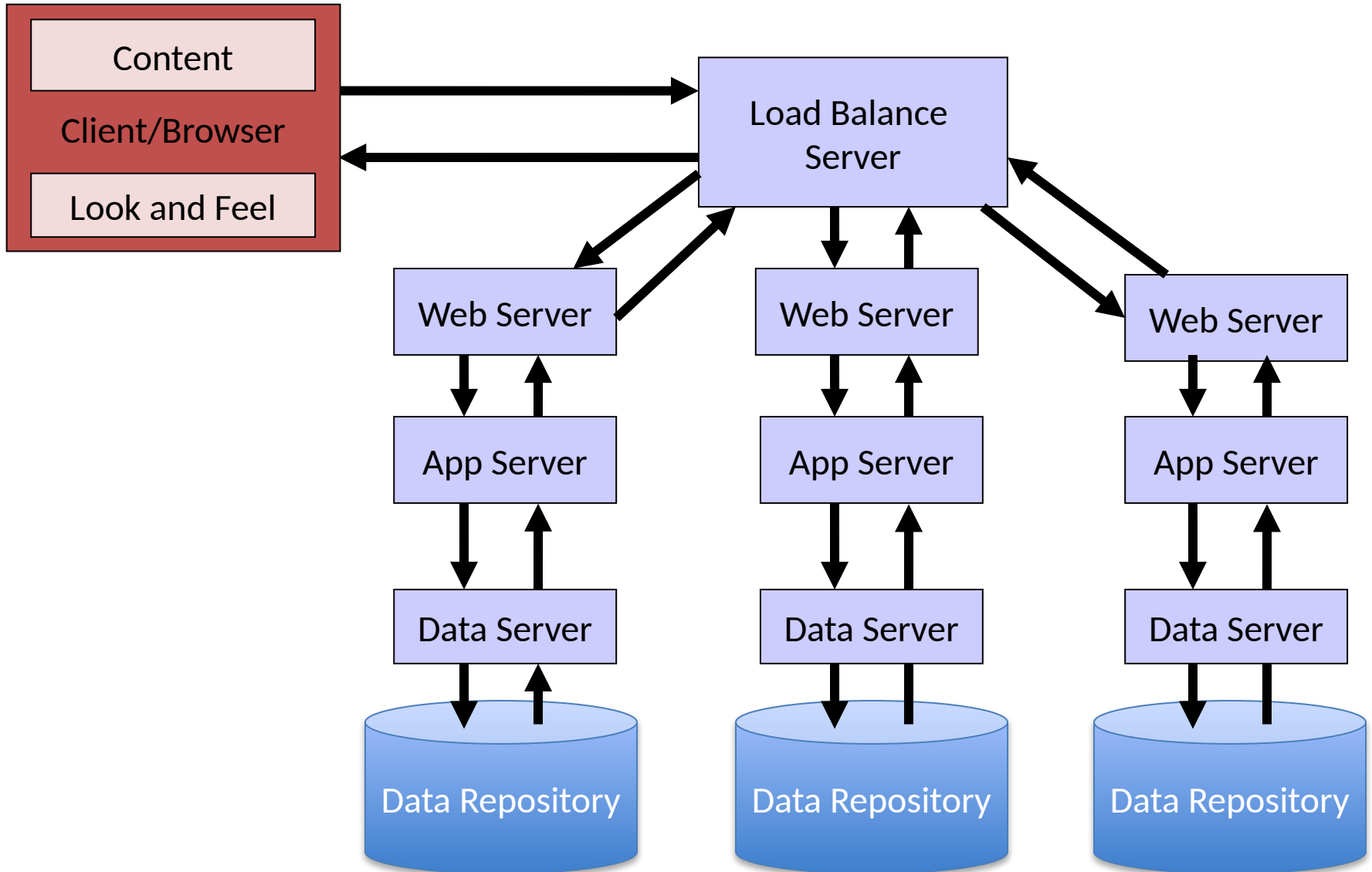
Within the middleware tier, usually there is a webserver, an application server, and potential media servers.

Webserver: handles incoming responses, directing them to the appropriate server.

Servers could be on the same machine or different machines.



N-Tier Architecture



Load Balancing

- There are two main ways to balance the load
 - Using a **Domain Name Server** such that when a URL is resolved, it rotates through a series of IP addresses that route the message to that machine.
 - Using a **Load Balancing Server**, which farms out the requests to available machines.
 - Machines in the farm inform the LB Server of their load so that the load can be balanced.

Tiered Architecture Benefits

- Tiers enable the separation of concerns
- Tiers encapsulate complexity
 - they can be broken down into layers or into sub-tiers
- Tiers can be distributed across a number of machines
 - Providing flexibility
- Tiers can be replicated across a number of machines
 - Providing scalability

Group Registration: UPDATE

- Groups for this course will now be **assigned automatically**
- **Self-formed groups won't be used**
- If you've already formed a group, **you may not stay in the same one**
- Your assigned group will be posted on **Moodle**

This change follows a **School-level decision** to make group work fair and consistent across courses.

Sorry for any inconvenience this causes, and thanks for your understanding 😊

SYSTEM ARCHITECTURE DESIGN

Top Down Design

Starting from a **high level design** is useful because:

- it helps to describe the system at a level which makes the **goals, scope** and **responsibilities** clear
- the abstraction provides a tool for communicating the design
- permits the specific technologies to be **chosen late** or to be **changed**
- **maintenance** and **re-usability** also profit
- **accessibility** variants can be considered *a priori*

Bottom Up Design

- Piecing together components to give rise to more complex systems, thus making the original elements sub-systems of the **emergent system**
- Specific and basic individual components of the system are first developed
- These elements are then linked together to form larger subsystems, which then in turn are linked, sometimes in many levels, until a complete top-level system is formed
- This strategy often resembles a "**seed**" **model**, by which the beginnings are small but eventually grow in complexity and completeness.

Top-Down vs. Bottom Up

Separates the low level work from the higher level abstractions

Leads to a modular design

Development can be self-contained (tiered)

Emphasizes planning and system understanding

Coding is late, and Testing is even later

Skeleton code can show how everything integrates



Coding begins early and so Testing can be performed early

Requires really good intuitions to determine functionality of modules

Low level design decisions can have major impact on solutions

Risks integration problems – how do components link together

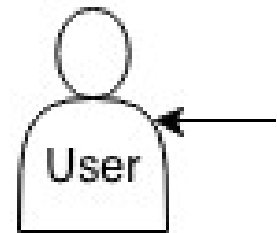
Often used to add on to existing modules

Notation for System Architecture Diagrams

- **Modified Dataflow Language** where we have the following entities:
 - User
 - Client
 - Middleware
 - Database/Datastore
 - Logs/Files
 - External Service/Application
 - Communications/DataFlows

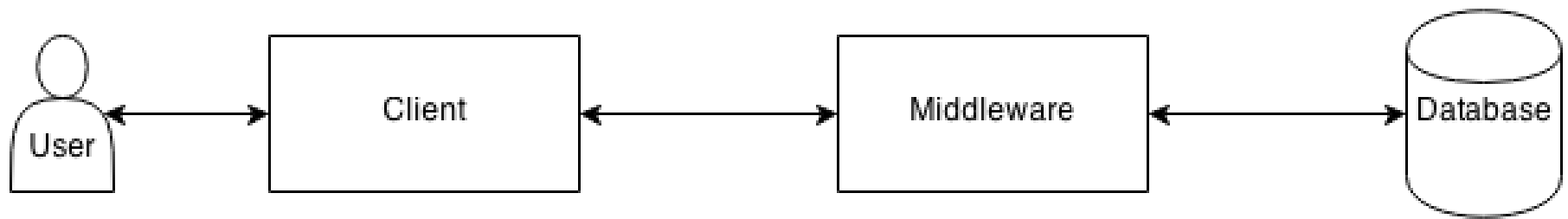
System Architecture Diagram: User

- The user or client instigates and interacts with the services or applications provided
- There are various clients including:
 - End Users (of varying abilities)
 - Administrators,
 - Developers,
 - Agents,
 - Other systems,
 - etc



System Architecture Diagram: Client

- The client and the interface presented takes on many forms and can vary greatly:
 - Web Browser on a PC, Tablet, Mobile, etc
 - An API for other systems, agents, developers, etc
 - Devices and Robots
 - Sensors

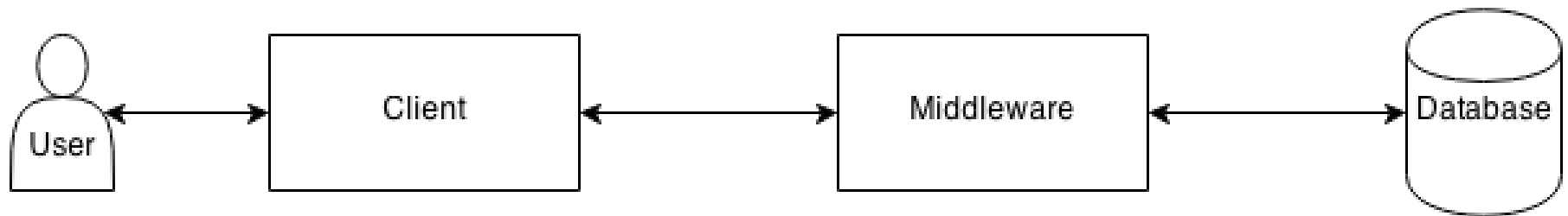


System Architecture Diagram: Middleware (1)

- The middleware houses an array of possible components from:
 - Domain Name Servers
 - Load Balancing Servers
 - Web Servers
 - Application Servers
 - Caching Servers
- Typically the first three are predefined or configured using standard software
- The Application Server is what is mainly of interest i.e. what needs to be developed.

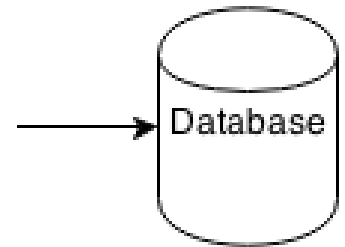
System Architecture Diagram: Middleware (2)

- Often represented as a single component that processes requests between the client and database.
 - Though encapsulates a number of other components



System Architecture Diagram: Databases

- A database server is usually employed to handle the data management side of applications.
 - i.e. Postgres, SQLServer, MySQL
- While the system is usually already in existence, it needs to be configured
 - i.e. the database tables have to be defined and populated.
 - To specify this part more precisely, ER Diagrams can be used.



System Architecture Diagram: Logs and External Services

- Logs represent data sinks
 - The application outputs data but does not read it back, directly.
- External Services represent applications and services that are used by the application
 - They provide an API or interface of some kind that which can be used to interact with the service.



System Architecture Diagram: Messaging

The **request message** is sent to the server from a user agent to:

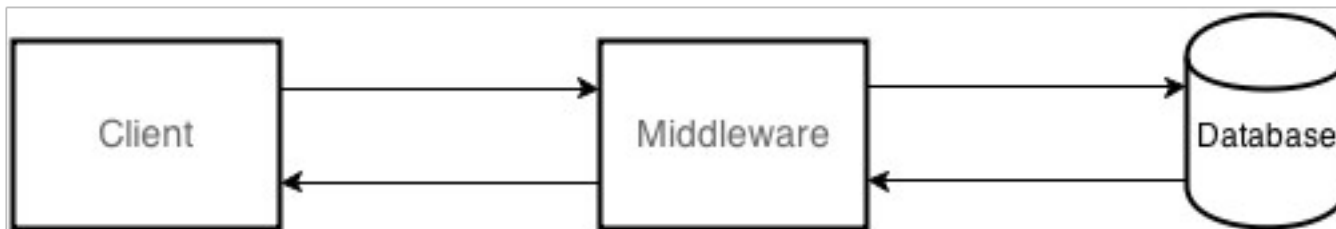
- ask for some information
- send some information to be stored
 - either from user input
 - or from some device (sensor)

The **response message** is sent from the server to a user agent to:

- return the requested information
- affect some change in the environment

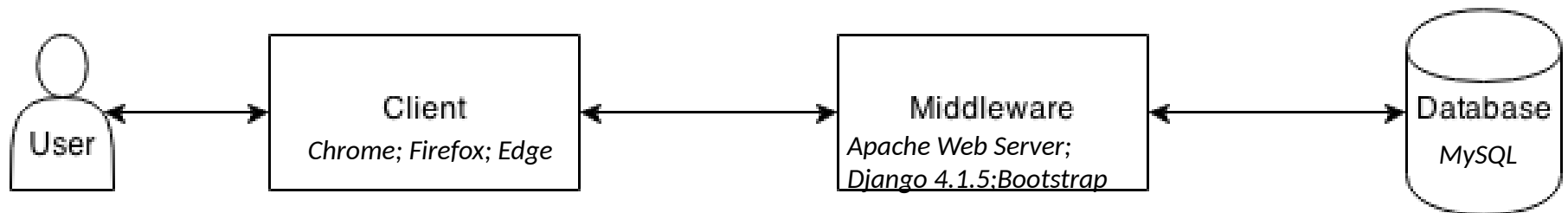
System Architecture Diagram: Data flows

- Arrows are used to denote the flow of information
 - The direction of the arrow denotes the direction of the communication.
 - Most communications are both ways, where a request is made, followed by a response.
 - The client makes the requests.
 - This show how the entities are related.



System Architecture Diagram: Technology and Devices

- For each box, we can state/specify the technology/device used, e.g.
 - Client: Web browser on a mobile device, using HTML/CSS/JS
 - Middleware: Apache Web Server, with an Application Server built using Django
 - Database: MySql Database Server



DATABASE MODELLING

Entity Relationship Model

- Provides an abstract representation of the data and how it relates to each other
 - Developed by Peter Chen in 1976
- Lend themselves to being implemented in a database (esp. relational databases)
- Three main components:
 - Entities,
 - Relationships and
 - Attributes

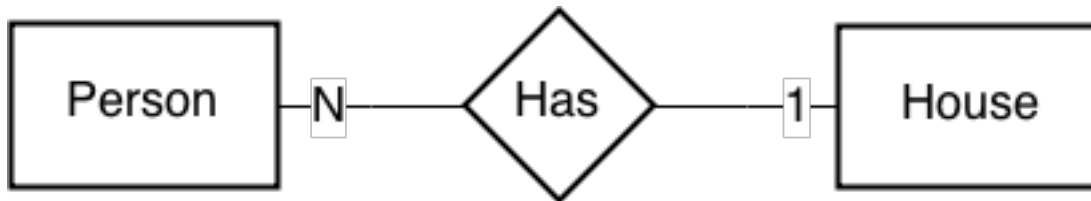


Notations

- Lots of different notations
 - Chen's
 - Bachman's
 - Barker's
 - Martin's
 - Etc
- Each support different ways to draw the ER model
 - We will be using a **Compressed Chen Notation**

Example

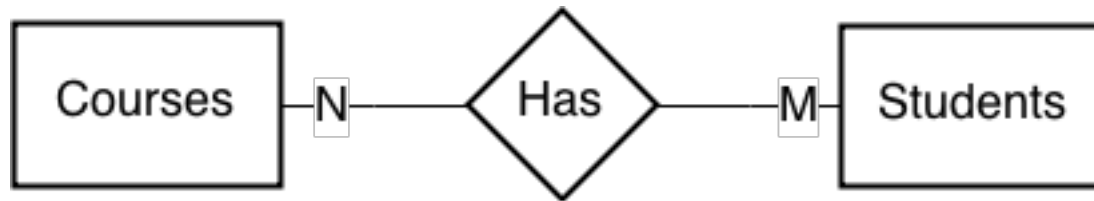
- Many people live in one house



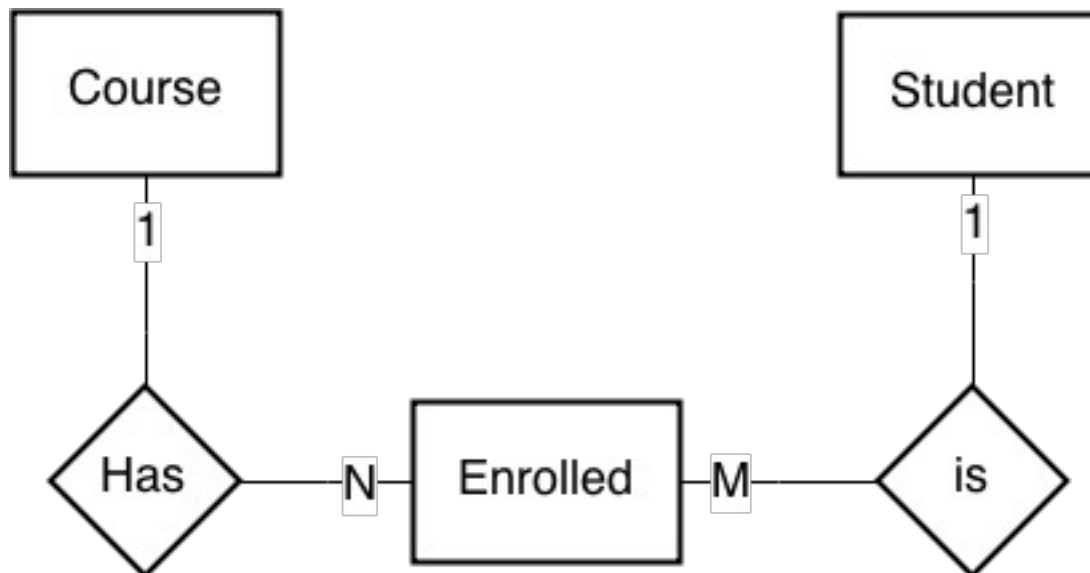
- **Rectangles** represent Entities
- **Diamonds** represent Relationships
- **1, N or M**, represents the cardinality of the relationship
 - N and M mean Many (usually you put M)
 - Different notations use crows feet, etc.

Many to Many

- Many courses are taken by many students
 - Can be represented like:

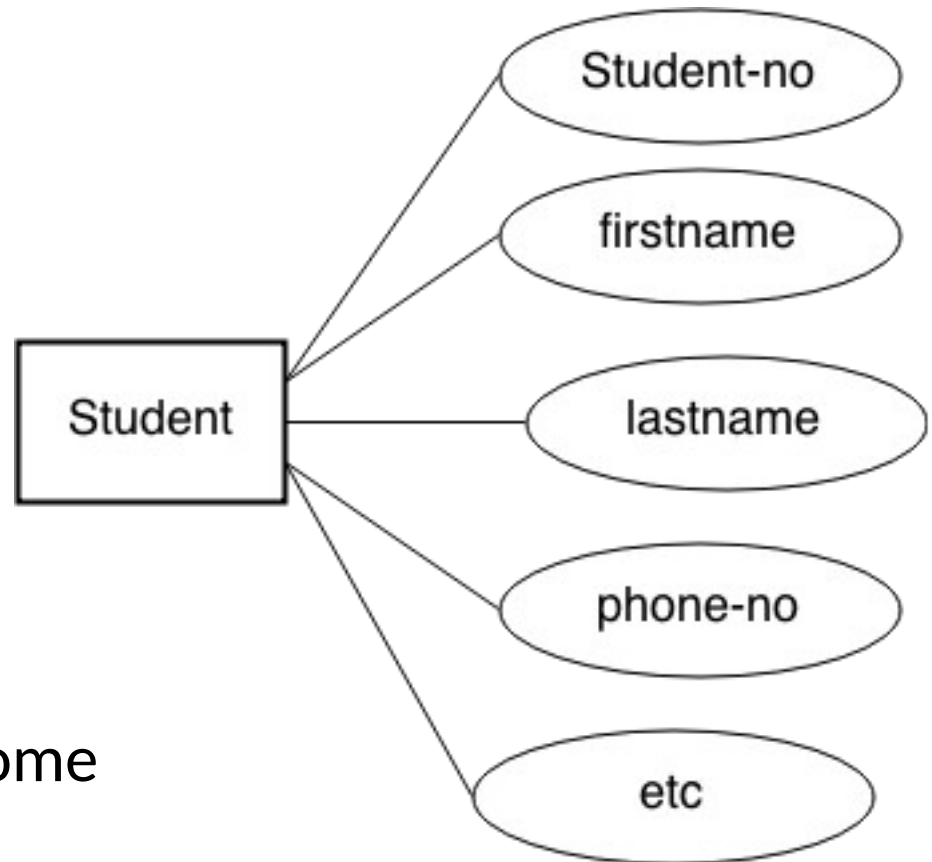


- Or like this:



Chen vs Compressed Chen (1)

- Chen Notation shows attributes as circles/ellipses



- This gets pretty cumbersome

Chen vs Compressed Chen (2)

- In compressed Chen notation, we only put entities and relationships in the diagram and then separately list the attributes

STUDENT	
Field	Type
Student No	Integer
Firstname	Char (128)
SecondName	Char (128)
Phone-no	Char (15), formatted
...	...

- This way we can neatly represent the attributes and their types

Converting to Django Models

- As a rule, each entity becomes a model.
- Then the relationship between the entities is specified using:
 - `models.ForeignKey` - denotes one-to-many
 - `models.OneToOneField` - denotes one-to-one
 - `models.ManyToManyField` - denotes m-to-n
- See TWD Chapter 5 and <https://docs.djangoproject.com/en/1.10/topics/db/examples/> for more details

INFORMATION ARCHITECTURE

Information Architecture

1. The **structural design** of shared information
2. The combination of **organization, labeling, search and navigation systems** within web sites and intranets
3. The **art and science** of shaping information products and experiences to support **usability** and **findability**.
4. An emerging discipline and community of practice focused on bringing **principles of design and architecture** to the **digital landscape**.

More: <http://www.uxbooth.com/articles/complete-beginners-guide-to-information-architecture/>

Thinking like a user

USER STAGES IN RECRUITMENT PROCESS FOR MEMP



THE POWERLESS PROSPECTIVE

- Has decided school/degree is a good option and is now seeking program distinctions. We need to answer: **How is Duke different?**
- Another possible entry point if step one was completed at another site. We are likely to rely on Google hits from this user.
- Asking about ROI and likely weighing reputation/cost between each institution.
- Is not yet sold on Duke and could possibly be swayed to other degree programs (MBA, Sciences, etc.) We need to reinforce why eng. mgmt. is a good direction.



THE APPLICANT

- Has decided that Duke and engineering management are good decisions and wants to apply.
- Is concerned with student life, assimilation for int'l, has lots of questions about logistics and qualifications.
- Might want to visit either virtually or in person.
- Is concerned with curriculum and collaboration w-univ. & v-globe.
- Possibly apply to 3-5 other institutions, some of which may not be eng. mgmt.



THE POWERFUL PROSPECTIVE

- Has been accepted to the program, and needs to decide whether to enroll.
- Asking again "is Duke right for me?" but with the confidence that "I'm right for Duke."
- Strongly weighing ROI, industry ties, curriculum reputation.
- Unless we provide some other information, user has likely seen it all before.



THE ENROLLED NEWBIE

- Has decided to enroll at Duke and has questions about moving to NC, courses to take.
- Is possibly interested in social implications. Might already be familiar with Duke/Durham.
- Might want to start relationships with faculty or Fugua.
- Already a potential recruiter for MEMP.

THE

- User even
- Send pas
- war for
- Pri bla
- Int

An Information Architect

- Assists to **identify user-based requirements.**

-  Is responsible for **how users interpret and interact with information.**

 - freeing up visual designers to concentrate on visual design elements and programmers to concentrate on code.

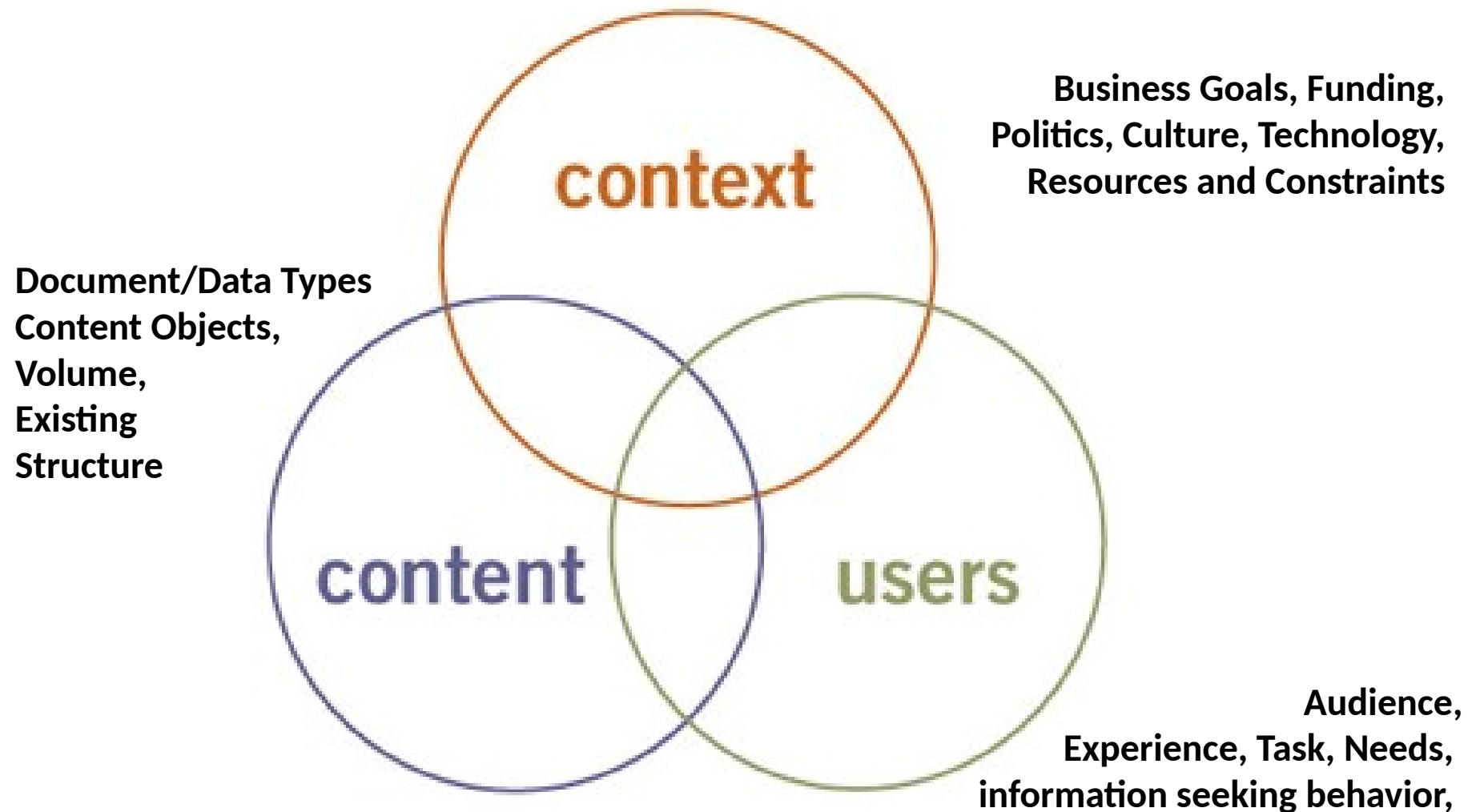
-  Investigate **customers and their needs**, factor **business strategy and technology resources** into **solutions.**

 - This is done long before programming begins

 - Can minimize loss of business and wasting of resources

 - Can increase and maintain revenue from customers

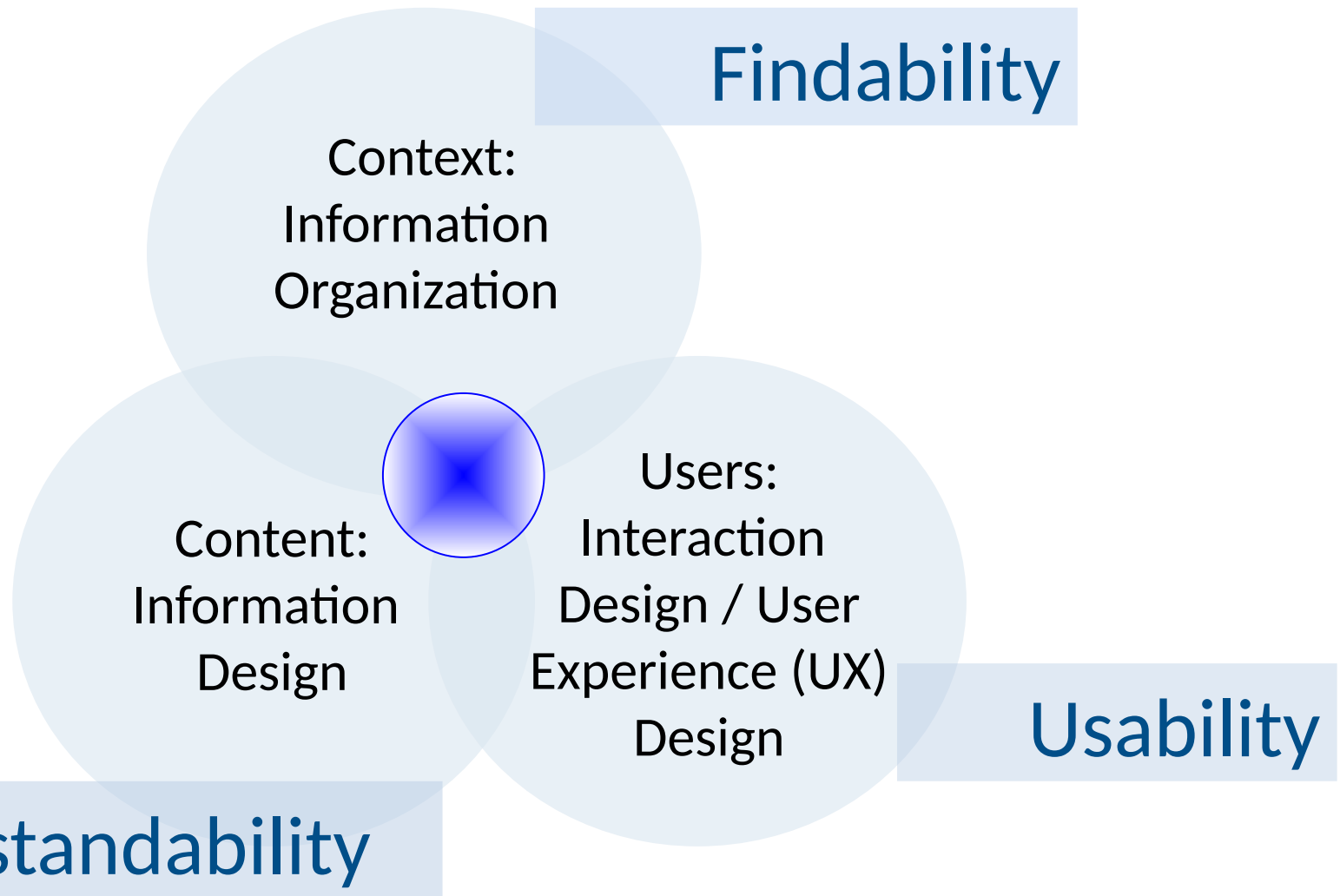
Context, Content & Users



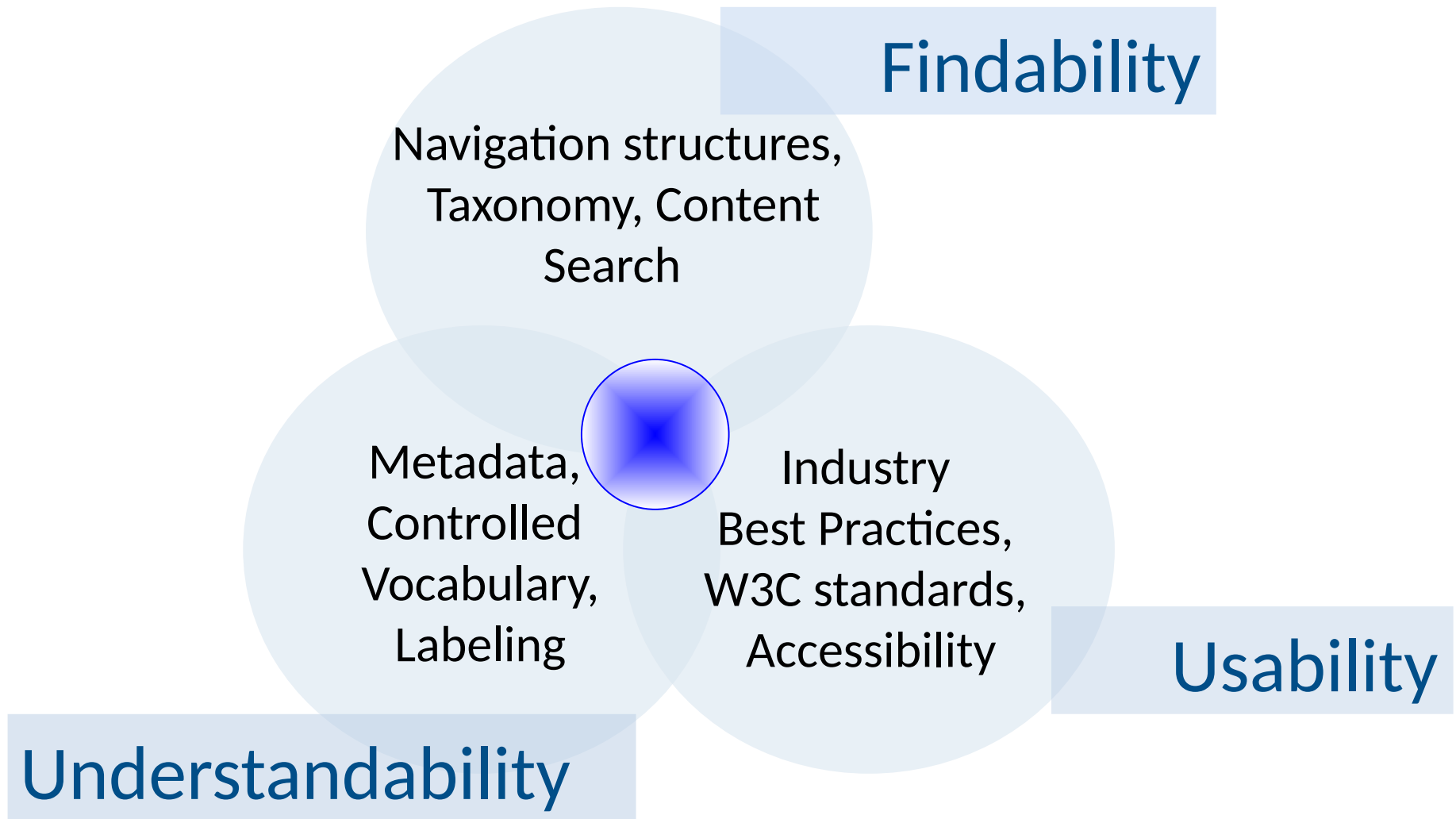
Rosenfield and Morville (the polar bear book):

<https://www.oreilly.com/library/view/information-architecture-for/0596527349/>

Developing Solutions



IA Components



User Driven Approach

- The fundamental questions an IA seeks to answer is:
 - Who is the user?
 - What do they need?
 - What will they see?
 - How will they interact with the system?
 - How will they get value from, accomplish their goals/tasks, etc?

IA Deliverables

- To answer such questions, IA have a number of tools and techniques that they employ, these include:
 - Specifying user requirements
 - Showing the structure of the site through site maps
 - Showing the sequence of interaction through user flows
 - Mocking up Wireframes to show structure of screens/pages

Specifying User Requirements

- Many ways to specify user requirements
 1. Simple list of requirements from the point of the user / User stories

*As a < type of user >, I want < some goal > so that
<some reason >.*
 2. Prioritized "User Needs" matrix
 3. Personas

User Stories

Example Website: University Café Menu website

- As a <student>, I want < to see today's menu> so that <I can decide what to eat before going to the café>.
- As a <student>, I want < to see the café's opening hours> so that <I know when it's open>.
- As a <café manager>, I want <to change opening hours> so that <students know when it is open>
- As a <café manager>, I want <to add or remove items quickly> so that <daily options can be displayed>

User Needs Matrix

User Groups and Roles >				Group 1: Treasury (T)			
				Overall Project Priority	Internal	External	
				Treasurer	Tax Administrator	Pricing Coordinator	Accounting Analyst (Finance & Expenses)
I want to...							
Analyze SEC Yields				2			3
Analyze Daily Yields				2			2
Review Monthly Report Card				4		5	
View Income & Distributions				2		2	1
Monitor Corporate Actions				4			3
Analyze Fund Performance				3		3	3
I INVESTMENT SUPPORT Affirm analysis							
T Tax Oversight/Analysis				3		2	
Determine loss carry forwards				3		3	
Manage Unrealized gain/loss				2		2	

A document to **capture user needs** of various **site users** and **prioritize** them accordingly.

- Ensures that all user requirements are captured
- Helps in prioritizing user needs and therefore helps in prioritizing page elements
- Creates a snapshot view of the user eco-system

Personas

User archetypes to help to guide decisions about product features, navigation, interactions, and even visual design



Roy Wechsler
Treasurer

General Description
As the head of the Treasury Department at SpringTide, Roy provides fund and tax oversight for a group of funds. He ensures that the funds comply with their prospectuses and any relevant regulations.

He has ultimate responsibility for the accounting of funds, including reconciliation between cash and custody, and reporting to the Board of Directors and the SEC.

Key Functions

- Fund oversight
- NAV monitoring
- Expense monitoring
- Tax oversight
- Compliance monitoring

Critical Needs

- Evaluate NAV against benchmarks
- Manage unrealized gain/loss
- Analyze fund compliance tests
- View income and distributions

Opportunities to improve user experience

- Provide data to support decision making
- Provide aggregate views across funds
- Support expense forecasting needs

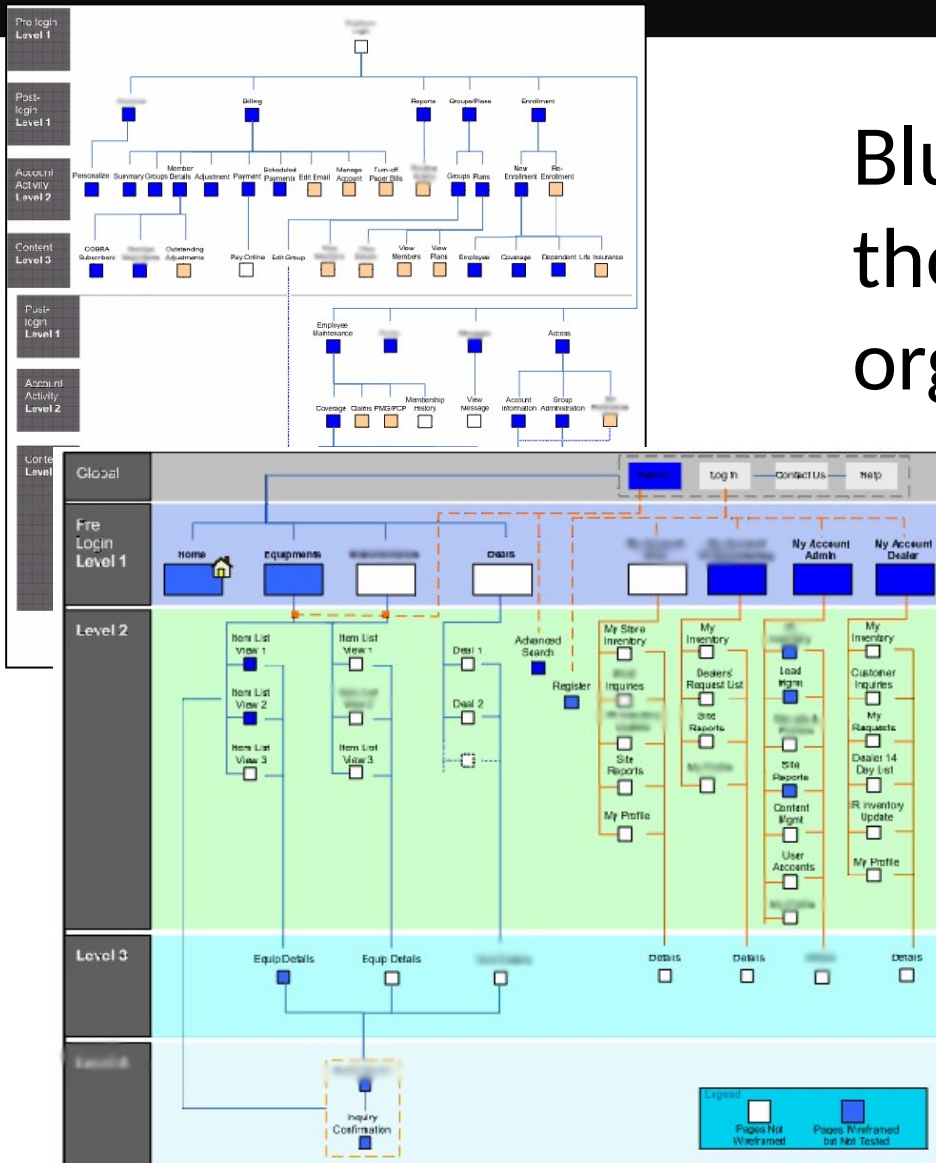
- **Demographics (User group)**
- **Needs (Goals)**
- **Environmental (Context)**

Prioritizing User Requirements

- **MoSCoW method**
 - **Must have:** essential for the primary user scenario (no workaround)
 - Example: sign up/log in; create/edit core item; view list/detail; input validated & saved
 - **Should have:** very important, but a temporary workaround exists
 - Example: search/filter; password reset; userprofile settings
 - **Could have:** nice-to-have if time allows
 - Example: dark mode, social sharing
 - **Won't have (this iteration):** explicitly out of scope for now
 - Example: real-time chat

Site Maps

Blueprints that shows how the site is going to be organized (sections/URLs).



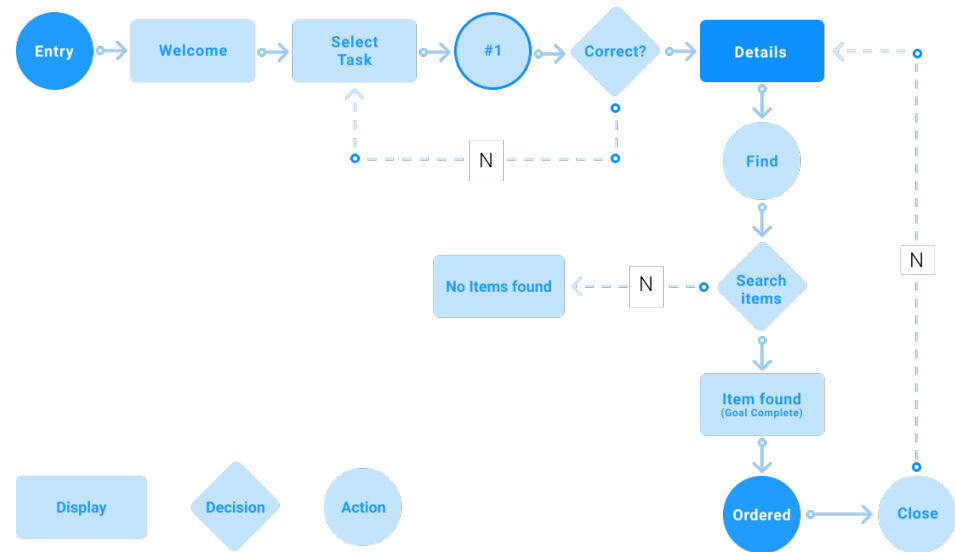
- Provides the high level snapshot view and relationship between pages of the site.
- Helps in restructuring the deep hierarchies.
- Helps to figure out navigation.

URL Design Tips

- **URLs should be obvious, and inferable**
 - URLs should use keywords, where possible
 - Shorter URLs are better
 - Try to avoid too many folders/depth
 - Use lowercase characters and avoid special characters in URLs
- **Use static URLs when you can**
 - This lets users revisit the information/page
 - Means that crawlers can index the content

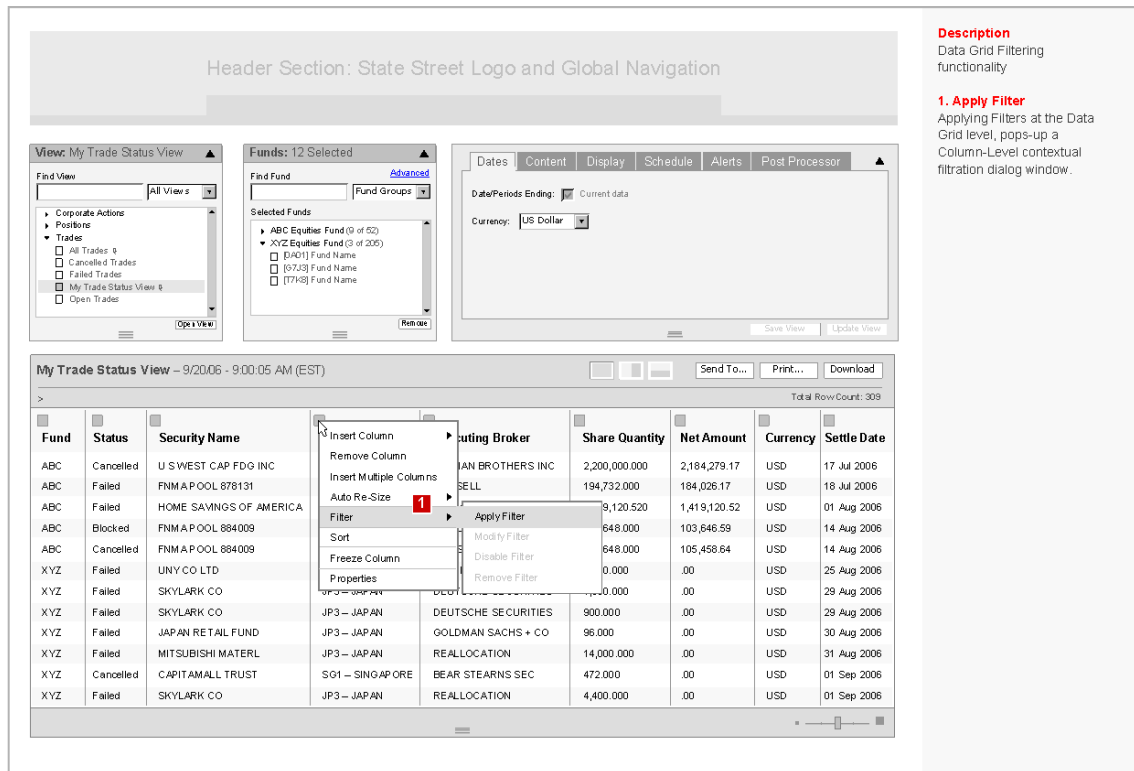
User flows

- Diagram that shows how a user moves through the site accomplishing a task
- Shows what a user has to accomplish to achieve a goal
- Let's you reduce complexity of interaction



Wireframes

Depict how an individual page or template will look from an architectural perspective. They are the intersection of the site's IA and its visual and information design.



- Saves lot of time
- Ideas can be presented without coding
- Helps in validating page elements and structure with the user.

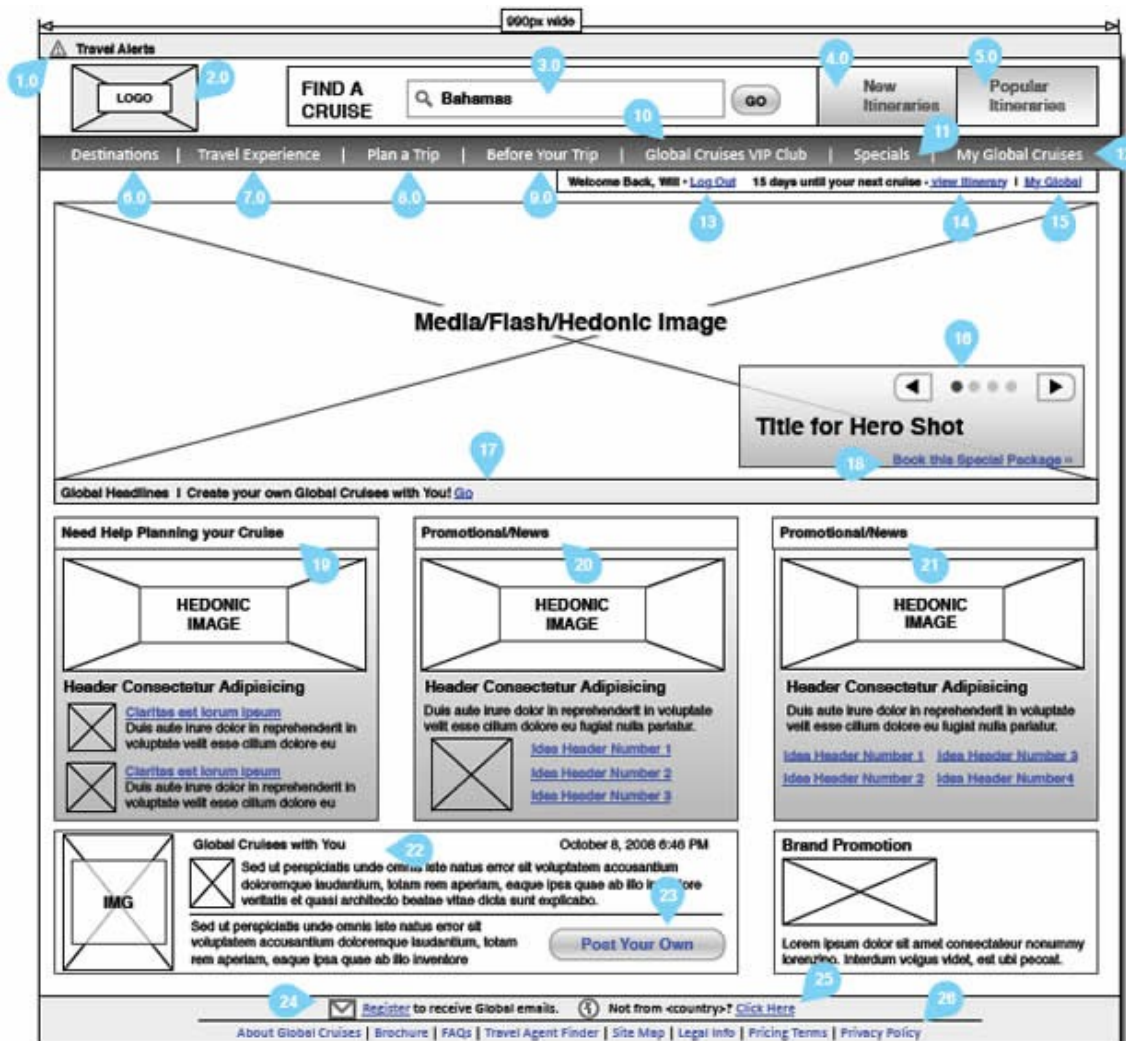
Drawing Wireframes (1)

- **Start by Sketching**
- **Iterate!**
- **Focus on Communication**
 - Keep the wireframe simple, abstract, but representative.
 - Number/label components
 - Explain the functionality of each component
 - Map features to system specifications/requirements

Drawing Wireframes (2)

- **Consider using a grid layout**
 - Determine layout of the main components with boxes
- **Wireframe with the team in mind** – what can be successfully developed?
- **Use real content** – to show **clients** what to expect and what they will see

Example specification wireframe



Annotated Notes	
1.0	Travel Alerts: link through to 0.2.0.0
2.0	Branding/Logo links Home Page
3.0	Search with predictive suggestion defined user scenario 3.X
4.0	New Itineraries drop down with link: display Itinerary w/link to section 4.x
5.0	Popular Itineraries - dropdown showing top 5 most popular Itineraries
6.0	Destinations Link: goes to section X.0
7.0	Travel Experience Link: goes to section X.0
8.0	Plan a Trip Link: goes to section X.0
9.0	Before Your Trip Link: goes to section X.0
10	Global Cruises VIP Club Link: goes to section X.0
11	Specials Link: goes to section X.0
12	My Global Cruises Link: goes to section X.0
13	Logout Link: logs user out of session
14	View Itinerary Link: Goes to My Global/View my Itineraries page.
15	My Global Link: Goes to personalized page
16	Carousel of specials/packages image
17	Starring You Moment Crowdsourcing link
18	Link to Book Specials tied to the Image in the large Hero shot
19	Need Help Planning your Cruise
20/21	Promotional CTAs/Partner Promotions
22	Global Cruises With You
23	Post your own Starring Your Moments link to crowdsourcing page.
24	Sign up to receive emails
25	Change Country link
26	Global Footer Links

Tools for drawing wireframes

- Simplest: Neat drawings by hand
- More presentable: Powerpoint, Balsamiq, Draw.io
- Specialised tools: e.g. Axure RP
 - Also able to be interactive

Accessibility

- How can people with a disability access/use the web?
- A very overlooked topic in web design!
- A mixture of good design and coding
 - For example, always provide ALT text for images (even if it is empty)



- **Up to 20% of users have some form of disability**
- **Standards and guidelines to make web accessible**
- **Increasing focus on web applications rather than static web sites**

Summary

- Modern business and entertainment relies on web applications
- System Architecture
 - Tiered system architecture aligns well with SE principles
 - System architecture is designed and expressed through diagrams
- DB Modelling
 - Modeling through ERDs allows you to think about the data and its relationship
 - Allows you to implement them in a database
 - Can be directly translated to Django models
- Information Architecture
 - Specifies the web application through the eyes of the user
 - Addresses findability, usability and understandability
 - Uses a variety of tools and techniques to communicate the design